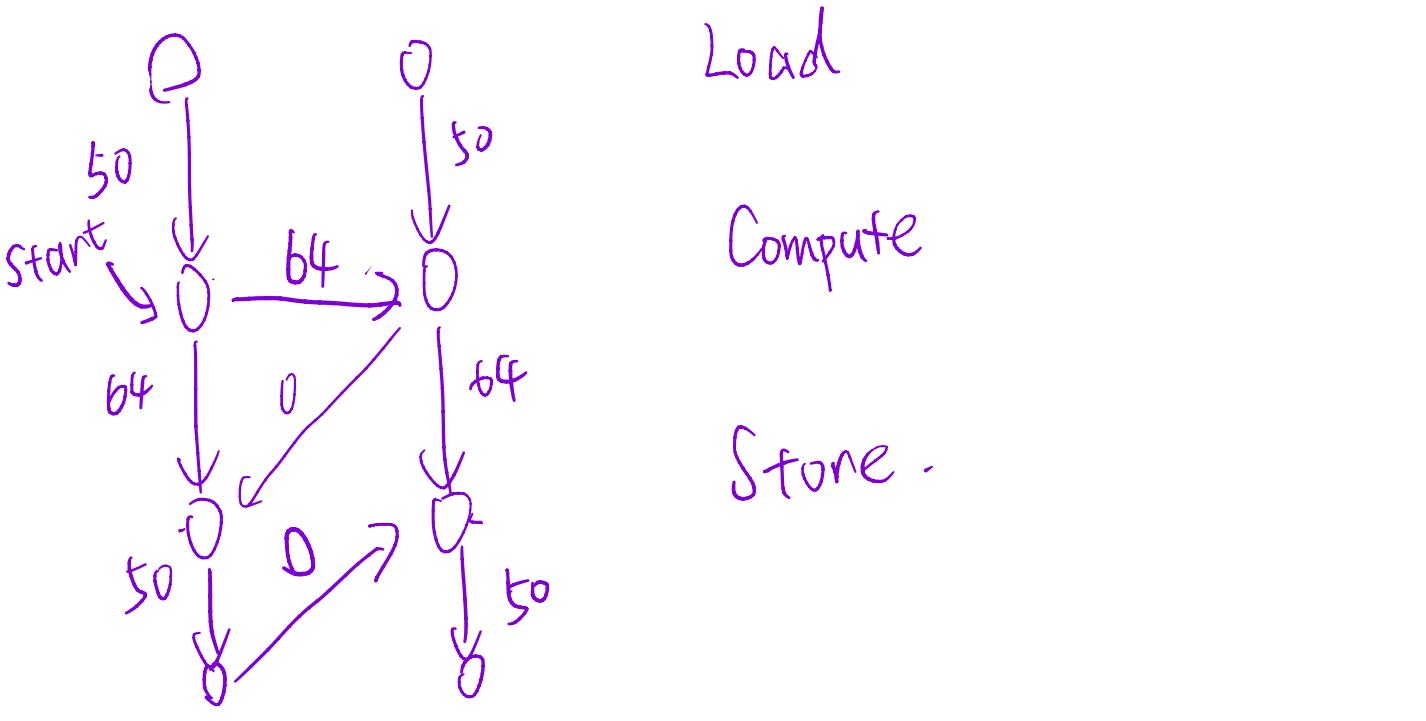
```python
@kdt.kernel(num_jobs_calculator=calculate_num_jobs)
def vector_add_kernel(task_args: Dict[str, int], io_tensors: Dict[str, kdt.Tile]):
    BLOCK_SIZE = 8192
    vec_size = task_args['N']
    num_blocks = vec_size // BLOCK_SIZE  # 计算需要处理的块数
    # 分配 SPM 上的数据块
    a_tile = kdt.alloc_spm((BLOCK_SIZE,), dtype='float32')
    b_tile = kdt.alloc_spm((BLOCK_SIZE,), dtype='float32')
    c_tile = kdt.alloc_spm((BLOCK_SIZE,), dtype='float32')
    d_tile = kdt.alloc_spm((BLOCK_SIZE,), dtype='float32')

    kdt.load(io_tensors['a'][0 * BLOCK_SIZE: (0 + 1) * BLOCK_SIZE], a_tile)
    kdt.load(io_tensors['b'][0 *  BLOCK_SIZE: (0 + 1) * BLOCK_SIZE], b_tile)
    for i in range(num_blocks // 2):
        j = i * 2
        # 加载输入数据到 SPM
        kdt.load(io_tensors['a'][(j + 1) * BLOCK_SIZE: (j + 2) * BLOCK_SIZE], c_tile)
        kdt.load(io_tensors['b'][(j + 1) * BLOCK_SIZE: (j + 2) * BLOCK_SIZE], d_tile)
        # 执行向量加法
        kdt.add(a_tile, b_tile, b_tile)
        kdt.add(c_tile, d_tile, d_tile)
        # 存储结果回显存
        kdt.store(b_tile, io_tensors['c'][j * BLOCK_SIZE: (j + 1) * BLOCK_SIZE])
        if j + 2 < num_blocks:
            kdt.load(io_tensors['a'][(j + 2) * BLOCK_SIZE: (j + 3) * BLOCK_SIZE], a_tile)
            kdt.load(io_tensors['b'][(j + 2) * BLOCK_SIZE: (j + 3) * BLOCK_SIZE], b_tile)
        kdt.store(d_tile, io_tensors['c'][(j + 1) * BLOCK_SIZE: (j + 2) * BLOCK_SIZE])
```

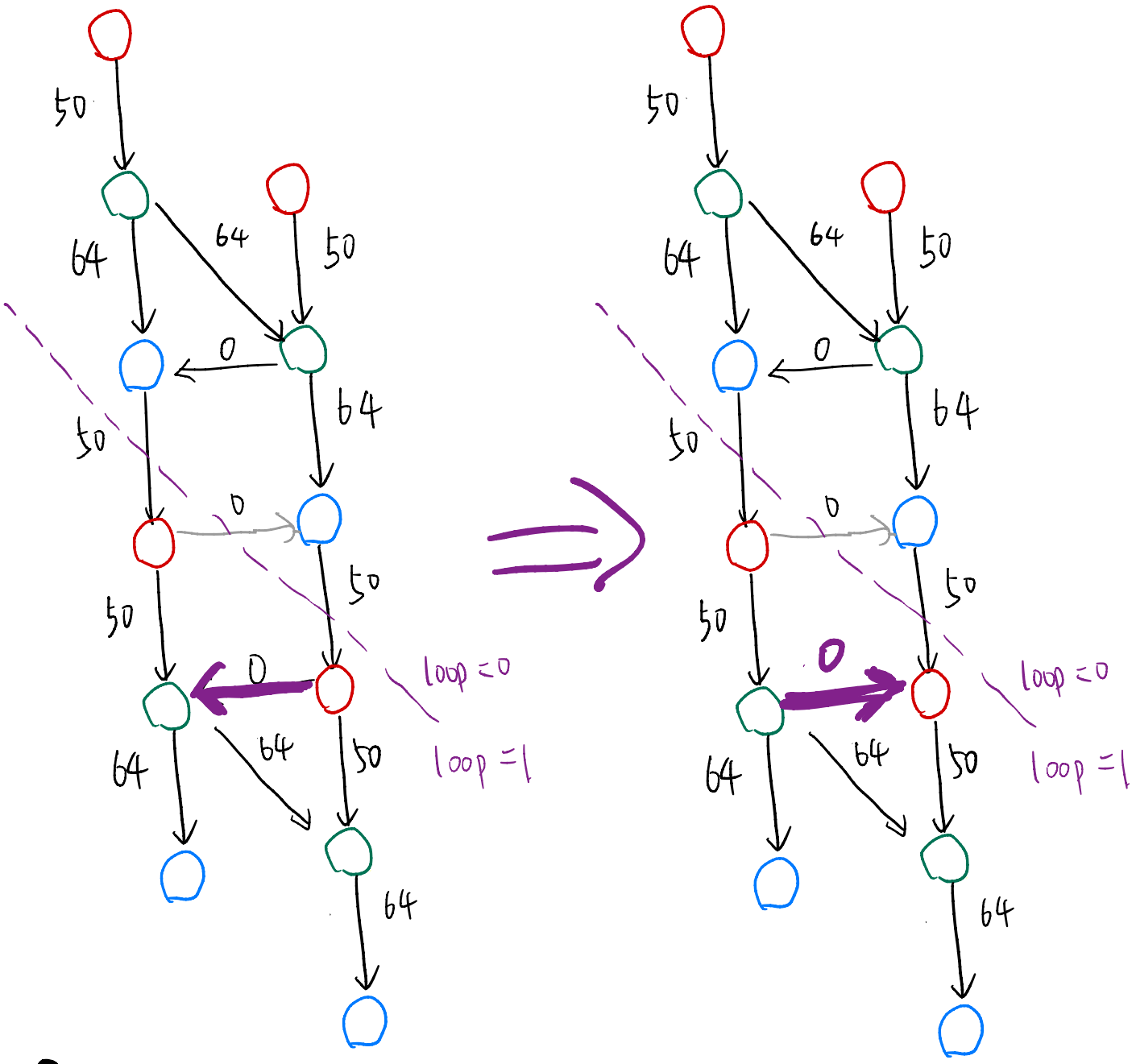
points : 9.3

每个结点代表指令的issue.



Load. Compute Store.

数据流与控制流图



Solution :

```
diff --git a/vector_add.py b/vector_add.py
index 0cc3c2a..60ade86 100644
--- a/vector_add.py
+++ b/vector_add.py
@@ -21,11 +21,12 @@ def vector_add_kernel(task_args: Dict[str, int], io_tensors: Dict[str, kdt.Tile]
     kdt.load(io_tensors['b'][0 *  BLOCK_SIZE: (0 + 1) * BLOCK_SIZE], b_tile)
     for i in range(num_blocks // 2):
         j = i * 2
+        # 执行向量加法
+        kdt.add(a_tile, b_tile, b_tile)
         # 加载输入数据到 SPM
         kdt.load(io_tensors['a'][(j + 1) * BLOCK_SIZE: (j + 2) * BLOCK_SIZE], c_tile)
         kdt.load(io_tensors['b'][(j + 1) * BLOCK_SIZE: (j + 2) * BLOCK_SIZE], d_tile)
         # 执行向量加法
-        kdt.add(a_tile, b_tile, b_tile)
         kdt.add(c_tile, d_tile, d_tile)
         # 存储结果回显存
         kdt.store(b_tile, io_tensors['c'][j * BLOCK_SIZE: (j + 1) * BLOCK_SIZE])
(END)
```

```
Total score: 5.6 / 100.0
(kdt-handout) ➜  kdt-handout git:(main) ✗ python3 judger/judger.py --kernel-impl-path vector_add.py --task 1
Running tests for Task 1...
Testcase 0 passed, num_cycles: 230, score: 100.0
Testcase 1 passed, num_cycles: 11507, score: 85.2
Task 1 (vector add) score: 9.3 / 10
Total score: 9.3 / 100.0
(kdt-handout) ➜  kdt-handout git:(main) ✗ python3 judger/judger.py --kernel-impl-path vector_add.py --task 1
Running tests for Task 1...
Testcase 0 passed, num_cycles: 230, score: 100.0
Testcase 1 passed, num_cycles: 10625, score: 100.0
Task 1 (vector add) score: 10.0 / 10
Total score: 10.0 / 100.0
```

```python

    kdt.load(io_tensors['a'][0 * BLOCK_SIZE: (0 + 1) * BLOCK_SIZE], a_tile)
    kdt.load(io_tensors['b'][0 *  BLOCK_SIZE: (0 + 1) * BLOCK_SIZE], b_tile)
    for i in range(num_blocks // 2):
        j = i * 2
        # 执行向量加法
        kdt.add(a_tile, b_tile, b_tile)
        # 加载输入数据到 SPM
        kdt.load(io_tensors['a'][(j + 1) * BLOCK_SIZE: (j + 2) * BLOCK_SIZE], c_tile)
        kdt.load(io_tensors['b'][(j + 1) * BLOCK_SIZE: (j + 2) * BLOCK_SIZE], d_tile)
        # 执行向量加法
        kdt.add(c_tile, d_tile, d_tile)
        # 存储结果回显存
        kdt.store(b_tile, io_tensors['c'][j * BLOCK_SIZE: (j + 1) * BLOCK_SIZE])
        if j + 2 < num_blocks:
            kdt.load(io_tensors['a'][(j + 2) * BLOCK_SIZE: (j + 3) * BLOCK_SIZE], a_tile)
            kdt.load(io_tensors['b'][(j + 2) * BLOCK_SIZE: (j + 3) * BLOCK_SIZE], b_tile)
        kdt.store(d_tile, io_tensors['c'][(j + 1) * BLOCK_SIZE: (j + 2) * BLOCK_SIZE])

```